2

« »

:

N3247

..

_____

()

:

..

_____

( )

_____

()

-

2026

1.

: (Radix Sort LSD) , , ; .

:

1. `enqueue`, `dequeue`, `queue_is_empty`, `queue_free`;
2. ;
3. LSD 10 -;
4. ;
5. ;
6. ;
7. - .

2.

2.1.

:

- `Node` `value` `next` .
- `Queue` `tail` ; `tail->next`. $O(1)$ (`enqueue`), (`dequeue`).
- : `node->next = node`.

:

`queue_init`		7
`queue_is_empty`	`size==0`	4
`enqueue`		30 ( ); 21 ()
`dequeue`		28 (.); 24 (.)
`queue_free`		$6n + 16$ ($n \geq 1$)

2.2. (Radix Sort LSD)

— , . LSD .

:

1. .

   `find_max` . d.

2. (`exp` = 1, 10, 100, ...).

   `max_val / exp > 0`, .

3. (`counting_sort_by_queue`):

   (a) **1.** 10 `buckets[0..9]`.

   (b) **2.** : `(arr[i] / exp) % 10`; `enqueue` .

   (c) **3.** `buckets[0..9]` , `dequeue` `arr[]`. FIFO .

   (d) **4.** 10 (`queue_free`).

:

- : $O(d \cdot n)$, n — , d — . d : $O(n)$.
- : $O(n)$ — n `Node`; .

7 (, , (`->`), , , ,). 1 ; .

$$T_{find\_max} = 7n - 2 \quad ()$$

`counting_sort_by_queue` ($k = 10$):

/	
1: j=0 (1); j<10 (11 .); j`++` (10×2); `&buckets[j]` (10); `queue_init` (10)	52
2: i ($3n$+2); : `arr[i]` + `/exp` + `%BASE` + `digit=` + `&buckets[digit]` + `arr[i]` + `enqueue` + `!=0` ($8 \times n$)	$11n + 2$
3: j=0 (1+1); j<10 (11 .); j`++` (10×2); `while is_empty` ($3 \times (n$+10)); `dequeue` ($3 \times n$); `idx++` ($2 \times n$)	$8n + 63$
4: j=0 (1); j<10 (11); j`++` (10×2); `&buckets[j]` (10); `queue_free` (10)	52
	19n + 169

$$T_1 = 19n + 169 \quad (k = 10)$$

$$T_{radix\_sort} = \underbrace{(7n - 2)}_{find\_max} + \underbrace{(7 + 5d)}_{} + d \cdot (19n + 169) = 7n + 5 + d \cdot (19n + 174)$$

: $T = O(d \cdot n)$, d $k = 10$ — $O(n)$.

2.3. (1)

	1 ()	**2** ()
	$26n + 120$	$19n + 169$
.	$O(n)$ + $O(k)$ count	$O(n)$ Node
- `malloc`	1	n
	( )	(FIFO)
	$O(d \cdot n)$	$O(d \cdot n)$

$(19n \;\; 26n)$, $(169 \;\; 120)$ - 10 . `malloc/free` , .

2.4.

1. : `input.txt`.
2. (`read_file`):
   - : n;
   - `malloc(n)`;
   - : .
3. (`print_array`).
4. **Radix Sort** (`radix_sort`).
5. .
6. : n, , d, k, , .
7. : `free(arr)`.

3.

1 .

1 –

(main.c)			
filename	`const char*`		
arr	`int*`		
n	`int`	$[1;\ 2^{31}-1]$	
max_val	`int`	$[0;\ 2^{31}-1]$	
d	`int`	$[1;\ 10]$	max_val
tmp	`int`	$[1;\ 2^{31}-1]$	
ops	`int`	$[0;\ 2^{31}-1]$	
(radix_sort.c)			
Node.value	`int`	$[0;\ 2^{31}-1]$	
Node.next	`Node*`		()
Queue.tail	`Node*`	/ NULL	; `tail->next` ==
Queue.size	`int`	$[0;\ n]$	
node	`Node*`		`enqueue`
head	`Node*`		`dequeue`
cur, tmp	`Node*`		`queue_free`
/ (main.c)			
fp	`FILE*`		
n	`int`	$[1;\ 2^{31}-1]$	
tmp	`int`	$[0;\ 2^{31}-1]$	
arr	`int*`		
i	`int`	$[0;\ n-1]$	
(radix_sort.c)			
exp	`int`	$\{1,\ 10,\ \ldots,\ 10^{d-1}\}$	
buckets[10]	`Queue[10]`	—	10 -
digit	`int`	$[0;\ 9]$	
idx	`int`	$[0;\ n-1]$	
i, j	`int`	$[0;\ n-1]$, $[0;\ 9]$	

max	`int`	$[0;\ 2^{31}-1]$	`find_max`
max_val	`int`	$[0;\ 2^{31}-1]$	

4.

4.1. 1: 20

input.txt:

```
170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5
```

```
  'input.txt'...
 20 .

 :
170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5

 Radix Sort...
 :
1 2 5 7 23 24 40 45 66 75 88 90 125 170 312 377 456 802 999 1000


  n ()            20
  max               1000
  d ()             4
  k (, BASE)      10
  :               2361
  .  ():    20
```

. : $7 \cdot 20 + 5 + 4 \cdot (19 \cdot 20 + 174) = 145 + 4 \cdot 554 = 2361$. — 20 Node ( ).

4.2. 2: 8

input.txt :

```
170 45 75 90 802 24 2 66
```

```
  'input.txt'...
 8 .

 :
170 45 75 90 802 24 2 66

 Radix Sort...
 :
2 24 45 66 75 90 170 802
```

```
n ()            8
max               802
d ()             3
k (, BASE)      10
:               1039
.  ():    8
```

: $7 \cdot 8 + 5 + 3 \cdot (19 \cdot 8 + 174) = 61 + 3 \cdot 326 = 1039.$

4.3. 3:

```
  'missing.txt'...
:     'missing.txt'
```

EXIT_FAILURE.

4.4. 4:

```
  'empty.txt'...
:  'empty.txt'
```

EXIT_FAILURE.

5. -

- (. 23).

-:

1. (`main`)
2. (`read_file`)
3. (`radix_sort`)
4. (`counting_sort_by_queue`)
5. (`enqueue`)
6. (`dequeue`)

6.

: main.c — , / ; radix_sort.h — ; radix_sort.c — .

6.1. main.c

```c
#include "radix_sort.h"
#include <stdio.h>
#include <stdlib.h>

/*
 *  read_file          .
 *   :   n,    .
 *  : 10n + 15  (   fscanf).
 */
static int read_file(const char *filename, int **out_arr)
{
    FILE *fp = fopen(filename, "r"); /* +1 , +1  */
    if (!fp) {                       /* +2  (!fp: fp == NULL,  !) */
        fprintf(stderr, ":     '%s'\n", filename);
        return -1;
    }

    /*  :   */
    int n = 0, tmp;                              /* +1  */
    while (fscanf(fp, "%d", &tmp) == 1) n++;    /* n+1 , n+1 ., n . */

    if (n == 0) { /* +1  */
        fprintf(stderr, ":  '%s'     \n", filename);
        fclose(fp);
        return -1;
    }

    rewind(fp); /* +1  */

    int *arr = (int *)malloc(n * sizeof(int)); /* +1 ., +1 , +1  */
    if (!arr) {                                /* +2  (!arr: arr == NULL,  !) */
        fprintf(stderr, ":    \n");
        fclose(fp);
        return -1;
    }

    /*  : i=0(+1 .); i<n(n+1 .); i++(n .) */
    for (int i = 0; i < n; i++) {
        if (fscanf(fp, "%d", &arr[i]) != 1) { /* +1 , +1 ., +1  */
            fprintf(stderr, ":   \n");
            free(arr);
```

```c
            fclose(fp);
            return -1;
        }
    }

    fclose(fp);    /* +1  */
    *out_arr = arr; /* +1  ( *out_arr), +1  */
    return n;       /* +1  */
}
/* : 10n + 15  */

/*
 *  print_array     .
 *  : 7n + 3 .
 */
static void print_array(const int *arr, int n)
{
    /* i=0: +1 .; i<n: n+1 .; i++: n . */
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);  /* +1 , +1  */
        if (i < n - 1)         /* +1 , +1  */
            printf(" ");       /* +1  */
    }
    printf("\n"); /* +1  */
}
/* : 7n + 3  */

int main(int argc, char *argv[])
{
    if (argc < 2) {                                  /* +1  */
        fprintf(stderr, ": %s <>\n", argv[0]); /* +1  */
        return EXIT_FAILURE;                         /* +1  */
    }
    const char *filename = argv[1];                  /* +1 , +1  */

    printf("\n  '%s'...\n", filename);    /* +1  */
    int *arr = NULL;                                 /* +1  */
    int  n   = read_file(filename, &arr);            /* +1 , +1  */
    if (n < 0) return EXIT_FAILURE;                  /* +1 , +1  ( ) */
    printf(" %d .\n", n);              /* +1  */

    printf("\n :\n");                  /* +1  */
    print_array(arr, n);                             /* +1  */

    printf("\n Radix Sort...\n");            /* +1  */
    radix_sort(arr, n);                              /* +1  */

    printf(" :\n");             /* +1  */
```

```
90      print_array(arr, n);                                    /* +1  */
91
92      /*  d        */
93      int max_val = 0;                                        /* +1  */
94      /* i=0(+1 .); i<n: n+1 .; i++(n): n .+n . */
95      for (int i = 0; i < n; i++) {
96          int v = arr[i] < 0 ? -arr[i] : arr[i];      /* +1 ., +1 ., +1 ., +1 . */
97          if (v > max_val) max_val = v;                /* +1 ., +1 . ( ) */
98      }
99      int d   = 0;                                            /* +1  */
100     int tmp = (max_val == 0) ? 1 : max_val;                 /* +1 ., +1 . */
101     /* d : d+1 . (tmp>0); d .+d . (tmp/=10); d .+d . (d++) */
102     while (tmp > 0) { d++; tmp /= 10; }
103
104     /* +1 .+1 .+1 .+1 . (7*n+5+d*(19*n+174)) */
105     int ops = 7*n + 5 + d*(19*n + 174);
106     printf("\n   \n"); /* +1  */
107     printf("  n ()           %d\n", n);    /* +1  */
108     printf("  max              %d\n", max_val); /* +1  */
109     printf("  d ()             %d\n", d);    /* +1  */
110     printf("  k (, BASE)     10\n");         /* +1  */
111     printf("  :              %d\n", ops);   /* +1  */
112     printf("  .  ():    %d\n", n);     /* +1  */
113     printf("\n"); /* +1  */
114
115     free(arr);           /* +1  */
116     return EXIT_SUCCESS; /* +1  */
117 }
```

main: ≈ 18 **+**

6.2. radix_sort.h

```
1  #ifndef RADIX_SORT_H
2  #define RADIX_SORT_H
3
4  int  find_max(const int *arr, int n);
5  void radix_sort(int *arr, int n);
6
7  #endif
```

6.3. radix_sort.c

```
81  /*
82   * queue_init
83   *  */
84  static void queue_init(Queue *q)
85  {
86      q->tail = NULL; /* +2  (->tail), +1  */
87      q->size = 0;    /* +2  (->size), +1  */
88  }                   /* +1    */
89  /* : 7  */
90
91  /*
92   * queue_is_empty   1
93   *  */
94  static int queue_is_empty(const Queue *q)
95  {
96      return q->size == 0; /* +2  (->size), +1 , +1  */
97  }
98  /* : 4  */
```

```
queue_init: 7 :  q->tail=NULL (2 +1 .)  + q->size=0 (2 +1 .)  + 1
```

```
queue_is_empty: 4 :  q->size==0 (2 +1 .)  + 1
```

enqueue

```
100 /*
101  * enqueue   value   , O(1)
102  *
103  *      :
104  *   malloc+.(2) + .(2)                          [node, !node: node==NULL +  !]
105  *   (2)+.(1)                                   [node->value=value]
106  *   (2)+.(1)                                   [q->size==0, false]
107  *   (2)+(4)+.(1)                          [node->next=q->tail->next]
108  *   (4)+.(1)                                   [q->tail->next=node]
109  *   (2)+.(1)                                   [q->tail=node]
110  *   (2)+.(1)+.(1)                          [q->size++]
111  *   (1)
112  *   = 30
113  *
114  *      :
115  *    (7+5): (2)+.(1)                       [node->next=node]
116  *   = 21
```

```
 *  */
static int enqueue(Queue *q, int value)
{
    Node *node = (Node *)malloc(sizeof(Node)); /* +1 , +1  */
    if (!node) {                               /* +2  (!node: node == NULL,  !) */
        fprintf(stderr, ": malloc  NULL\n"); /* +1  */
        return -1;                             /* +1  */
    }

    node->value = value; /* +2  (->value), +1  */

    if (q->size == 0) {              /* +2  (->size), +1  */
        node->next = node;           /* +2  (->next), +1  */
    } else {
        node->next    = q->tail->next; /* +2  (node->next), +4  (q->tail->next), +1  */
        q->tail->next = node;          /* +4  (q->tail->next), +1  */
    }
    q->tail = node; /* +2  (->tail), +1  */
    q->size++;      /* +2  (->size), +1 , +1  */
    return 0;       /* +1  */
}
/*  ( ): 30 ;  (): 21  */
```

```
enqueue: 30  (  ); 21  ()
```

dequeue

```
/*
 * dequeue     , O(1)
 *
 *     :
 *   (2)+.(1)                                  [q->size==0, false]
 *   (4)+.(1)                                  [head=q->tail->next]
 *   (1)+(2)+.(1)                          [*out=head->value]
 *   (2)+.(1)                                  [head==q->tail, false]
 *   (4)+(2)+.(1)                          [q->tail->next=head->next]
 *   (1)                                           [free(head)]
 *   (2)+.(1)+.(1)                          [q->size--]
 *   (1)
 *   = 28
 *
 *     :
 *    (4+2+1): (2)+.(1)                    [q->tail=NULL]
 *   = 24
 *  */
static int dequeue(Queue *q, int *out)
```

```
{
    if (q->size == 0) {                /* +2  (->size), +1  */
        fprintf(stderr, ":  \n"); /* +1  */
        return -1;                     /* +1  */
    }

    Node *head = q->tail->next;      /* +4  (q->tail->next), +1  */
    *out = head->value;              /* +1  (*out .), +2  (->value), +1  */

    if (head == q->tail) {           /* +2  (q->tail), +1  */
        q->tail = NULL;              /* +2  (->tail), +1  */
    } else {
        q->tail->next = head->next; /* +4  (q->tail->next), +2  (head->next), +1  */
    }
    free(head);  /* +1  */
    q->size--;   /* +2  (->size), +1 , +1  */
    return 0;    /* +1  */
}
/*  ( ): 28 ;  ( ): 24  */
```

```
dequeue: 28  ( ); 24  ( )
```

```
/*
 * queue_free
 *
 *  : (2)+.(1)+(1) = 4
 * n  1:
 *   (2)+.(1)                        [q->size==0, false] = 3
 *   (4)+.(1)                        [head=q->tail->next] = 5
 *   .(1)                                  [cur=head] = 1
 *    » n:
 *     .(1)                                 [tmp=cur]
 *     (2)+.(1)                        [cur=cur->next]
 *     (1)                                  [free(tmp)]
 *     .(1)                                 [cur!=head]
 *       : 6                      6n
 *   (2)+.(1)                        [q->tail=NULL] = 3
 *   (2)+.(1)                        [q->size=0] = 3
 *   (1)                                  = 1
 *  */
static void queue_free(Queue *q)
{
    if (q->size == 0) return; /* +2  (->size), +1 , +1  */

```

```
    Node *head = q->tail->next; /* +4  (q->tail->next), +1  */
    Node *cur  = head;          /* +1  */

    /* n  do-while */
    do {
        Node *tmp = cur;    /* +1  */
        cur = cur->next;    /* +2  (->next), +1  */
        free(tmp);          /* +1  */
    } while (cur != head);  /* +1  */

    q->tail = NULL; /* +2  (->tail), +1  */
    q->size = 0;    /* +2  (->size), +1  */
}                   /* +1  */
/*  (n  1): 6n + 16  */
```

queue_free: $6n + 16$ $(n \geq 1)$; **4** ( )

```
/*
 * find_max    , O(n)
 *
 * max=arr[0]:            .(1)+.(1)                  = 2
 * i=1:                   .(1)                          = 1
 * i<n  (n ):          .(1) » n                      = n
 * i<n  :    .(1)                          = 1  (  n)
 * i++ (n1 ):         .(1)+.(1) » (n1)       = 2(n1)
 * arr[i]>max (n1 ):  .(1)+.(1)                 = 2(n1)
 * max=arr[i] ():   .(1)+.(1) » (n1)          = 2(n1)
 * return:                (1)                             = 1
 * : 2+1+n+2(n-1)+2(n-1)+2(n-1)+1 = 7n  2
 *  */
int find_max(const int *arr, int n)
{
    int max = arr[0];   /* +1 , +1  */
    /* i=1(+1 .); i<n: n .; i++: (n-1)(+1 .+1 .) */
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)  /* +1 , +1  */
            max = arr[i];  /* +1 , +1  */
    }
    return max;            /* +1  */
}
/* : 7n  2  ( ) */

/*
 * counting_sort_by_queue      exp
```

```
 *
 *   1   10                    = 52
 *   2   (i = 0..n1)                 = 11n + 2
 *   3     (j = 0..9, dequeue » n)     = 8n + 63
 *   4   10                     = 52
 *
 *  : 19n + 169
 *  */
static void counting_sort_by_queue(int *arr, int n, int exp)
{
    Queue buckets[BASE];

    /*  1: j=0(+1 .); j<10: 11 .;
              j++(»10): 10 .+10 .;
              &buckets[j](+1 .), queue_init(+1 )  10
                1: 52 */
    for (int j = 0; j < BASE; j++)
        queue_init(&buckets[j]); /* +1 , +1  */

    /*  2: i=0(+1 .); i<n: n+1 .;
              i++(»n): n .+n .;
              arr[i](+1 .), /exp(+1 .), %BASE(+1 .),
              digit=(+1 .), &buckets[digit](+1 .),
              arr[i](+1 .), enqueue(+1 ), !=0(+1 .)  n
                2: 11n + 2 */
    for (int i = 0; i < n; i++) {
        int digit = (arr[i] / exp) % BASE; /* +1 , +1 , +1 , +1  */
        if (enqueue(&buckets[digit], arr[i]) != 0) { /* +1 , +1 , +1 , +1  */
            fprintf(stderr, ": enqueue  -1\n");
            exit(EXIT_FAILURE);
        }
    }

    /*  3: idx=0(+1 .); j=0(+1 .);
              j<10: 11 .; j++(»10): 10 .+10 .;
              while »(n+10): &buckets[j](+1 .), is_empty(+1 ), !(+1 .);
              dequeue »n: &buckets[j](+1 .), &arr[idx](+1 .), dequeue(+1 );
              idx++(»n): n .+n .
                3: 8n + 63 */
    int idx = 0; /* +1  */
    for (int j = 0; j < BASE; j++) {           /* +1  */
        while (!queue_is_empty(&buckets[j])) { /* +1 , +1 , +1  */
            dequeue(&buckets[j], &arr[idx]);   /* +1 , +1 , +1  */
            idx++;                             /* +1 , +1  */
        }
    }

    /*  4:   1
```

```
                4: 52 */
    for (int j = 0; j < BASE; j++)
        queue_free(&buckets[j]); /* +1 , +1  */
}
/* : 19n + 169  */

/*
 * radix_sort  LSD Radix Sort
 *
 *   ( ):
 *   n1(+1 .)
 *   find_max(+1 +1 .)
 *   exp=1(+1 .)
 *   max_val/exp>0 » (d+1): (d+1)(+1 .+1 .)
 *   exp*=BASE » d: d(+1 .+1 .)
 *   counting_sort » d: d
 *   return: +1
 *     : 7 + 5d
 *
 *  ( ):
 *   find_max:          7n  2
 *   counting » d:      d ů (19n + 169)
 *    :  7 + 5d
 *
 *   : 7n + 5 + d ů (19n + 174)
 *  */
/*
 * radix_sort_nonneg  LSD Radix Sort
 *  */
static void radix_sort_nonneg(int *arr, int n)
{
    if (n <= 1) return;
    int max_val = find_max(arr, n);
    for (int exp = 1; max_val / exp > 0; exp *= BASE)
        counting_sort_by_queue(arr, n, exp);
}

/*
 * radix_sort  LSD Radix Sort
 *
 * :
 *   1.   neg[] ( .)  pos[] (.)
 *   2.     radix_sort_nonneg
 *   3. : neg      ń-ż,  pos
 *  */
void radix_sort(int *arr, int n)
{
    if (n <= 1) return;
```

```
343
344     /*   */
345     int neg_n = 0;
346     for (int i = 0; i < n; i++)
347         if (arr[i] < 0) neg_n++;
348     int pos_n = n - neg_n;
349
350     /*           */
351     if (neg_n == 0) { radix_sort_nonneg(arr, n); return; }
352     if (pos_n == 0) {
353         /*  ,     */
354         for (int i = 0; i < n; i++) arr[i] = -arr[i];
355         radix_sort_nonneg(arr, n);
356         /*     */
357         for (int i = 0, j = n - 1; i < j; i++, j--) {
358             int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
359         }
360         for (int i = 0; i < n; i++) arr[i] = -arr[i];
361         return;
362     }
363
364     /*  :     */
365     int *neg = (int *)malloc(neg_n * sizeof(int));
366     int *pos = (int *)malloc(pos_n * sizeof(int));
367     if (!neg || !pos) {
368         fprintf(stderr, ": malloc  radix_sort\n");
369         free(neg); free(pos); exit(EXIT_FAILURE);
370     }
371
372     int ni = 0, pi = 0;
373     for (int i = 0; i < n; i++) {
374         if (arr[i] < 0) neg[ni++] = -arr[i]; /*  */
375         else            pos[pi++] =  arr[i];
376     }
377
378     radix_sort_nonneg(neg, neg_n);
379     radix_sort_nonneg(pos, pos_n);
380
381     /* :        */
382     int idx = 0;
383     for (int i = neg_n - 1; i >= 0; i--)
384         arr[idx++] = -neg[i];
385     for (int i = 0; i < pos_n; i++)
386         arr[idx++] = pos[i];
387
388     free(neg);
389     free(pos);
390 }
```

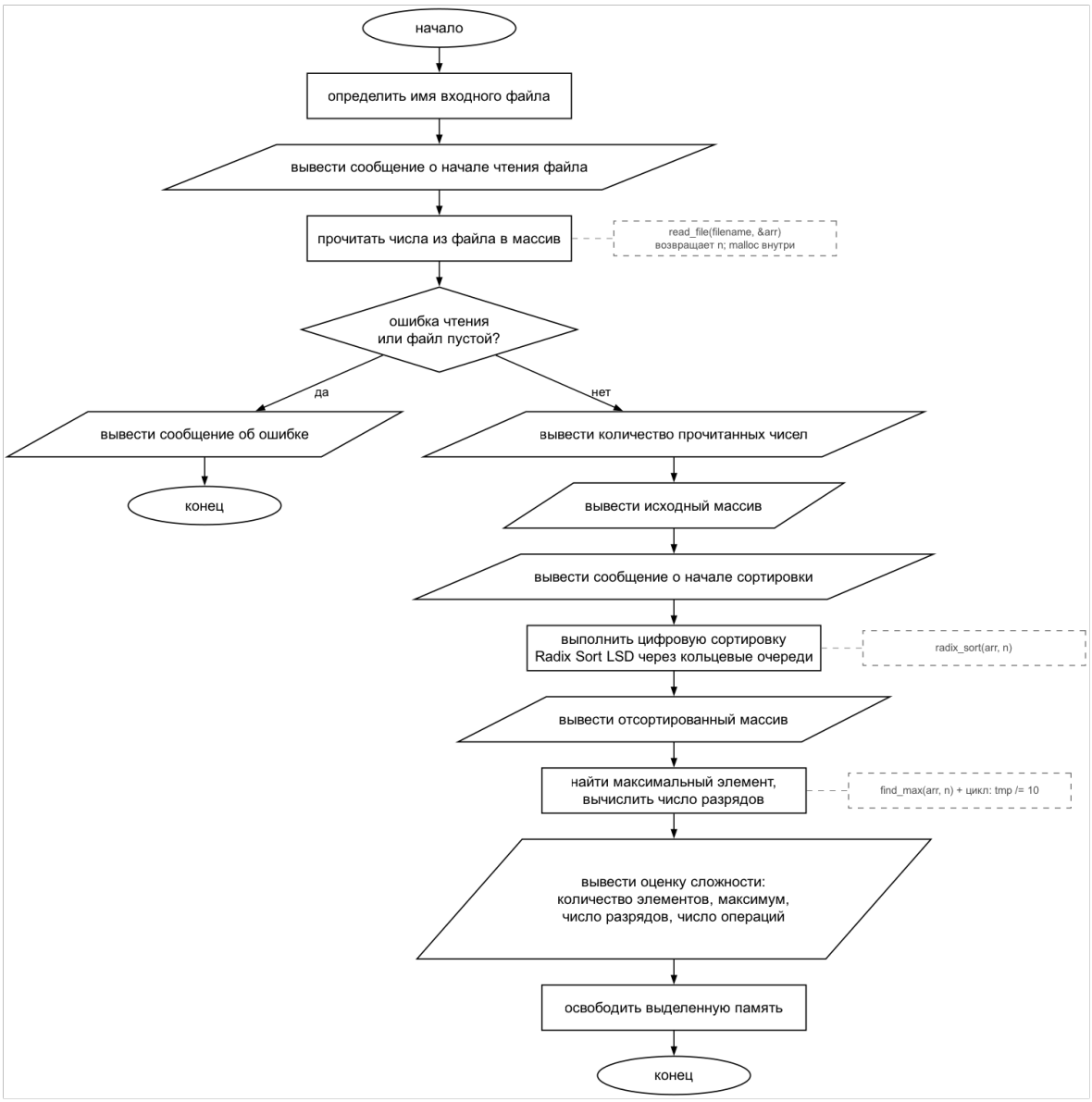
`find_max`: $7n - 2$ `( )`

`counting_sort_by_queue`: $19n + 169$ (`k`=10)

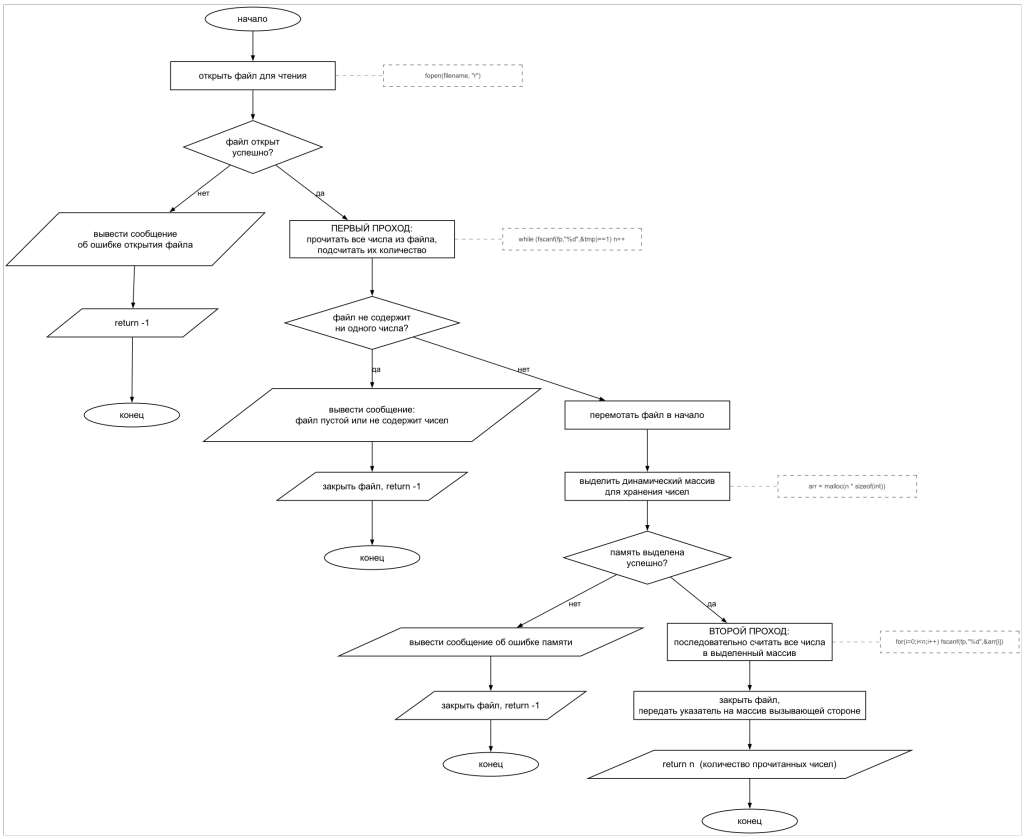
`radix_sort`: $7n + 5 + d \cdot (19n + 174)$

C, (Radix Sort LSD) .
: `main.c` , / ; `radix_sort.h` ; `radix_sort.c` Radix Sort.
`output[]` `count[]` 10 -: `enqueue`, `dequeue`. FIFO .
$O(d \cdot n)$ $O(n)$. d $O(n)$. (1, $26n + 120$) ($19n$ $26n$),
(169 120) - 10 . . .

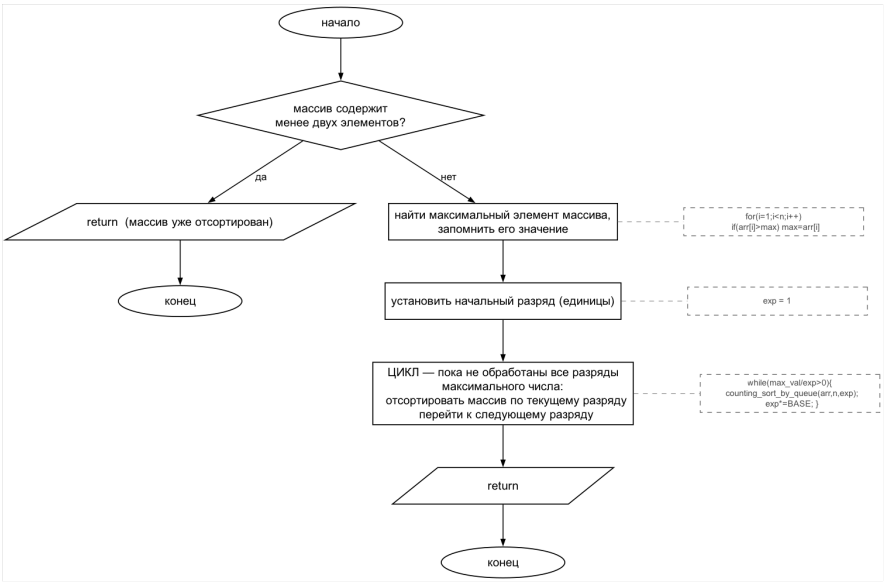
. -


начало
определить имя входного файла
вывести сообщение о начале чтения файла
прочитать числа из файла в массив
read_file(filename, &arr)
возвращает n; malloc внутри
ошибка чтения
или файл пустой?
да
нет
вывести сообщение об ошибке
конец
вывести количество прочитанных чисел
вывести исходный массив
вывести сообщение о начале сортировки
выполнить цифровую сортировку
Radix Sort LSD через кольцевые очереди
radix_sort(arr, n)
вывести отсортированный массив
найти максимальный элемент,
вычислить число разрядов
find_max(arr, n) + цикл: tmp /= 10
вывести оценку сложности:
количество элементов, максимум,
число разрядов, число операций
освободить выделенную память
конец

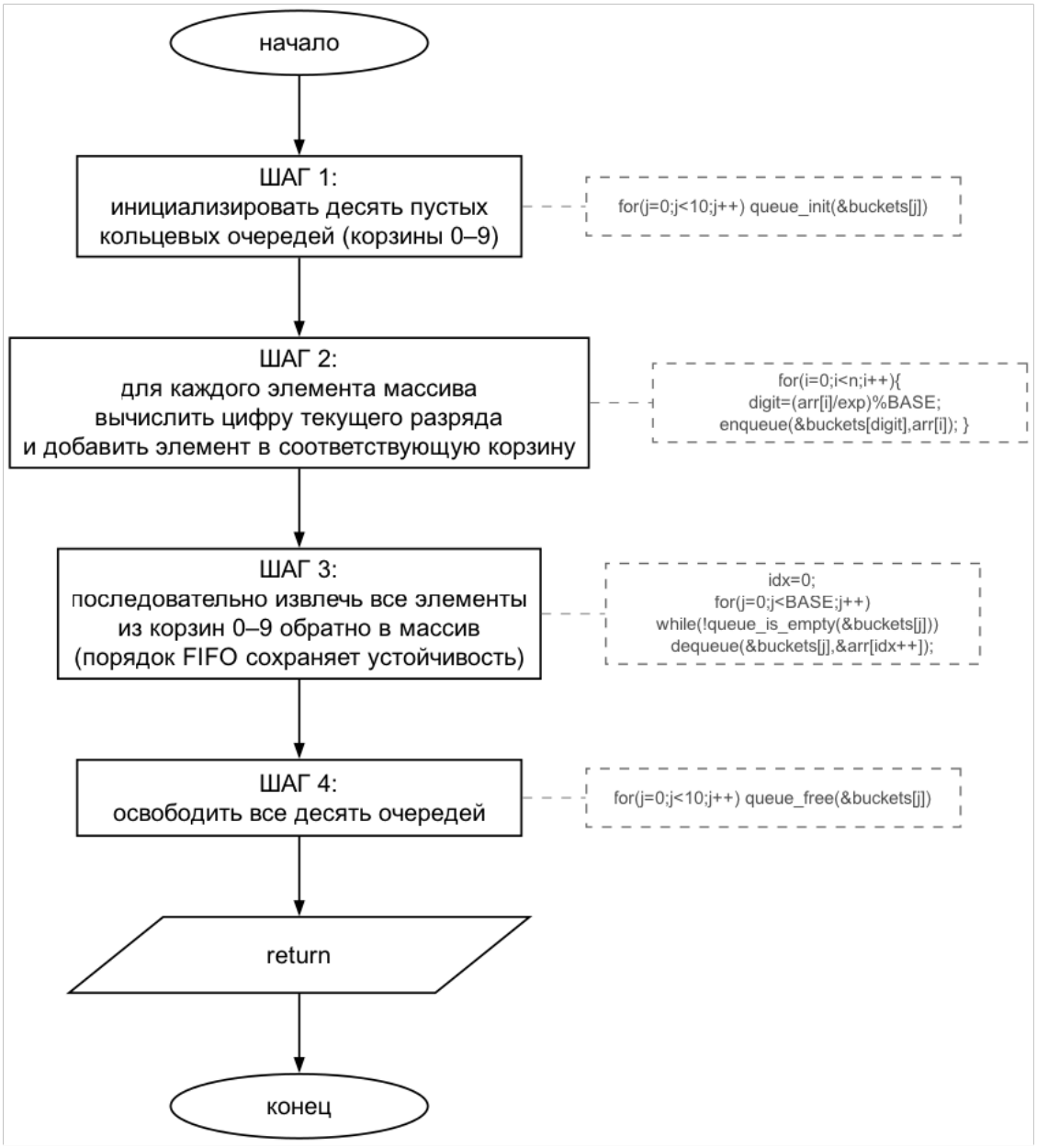
. 1 – (main)



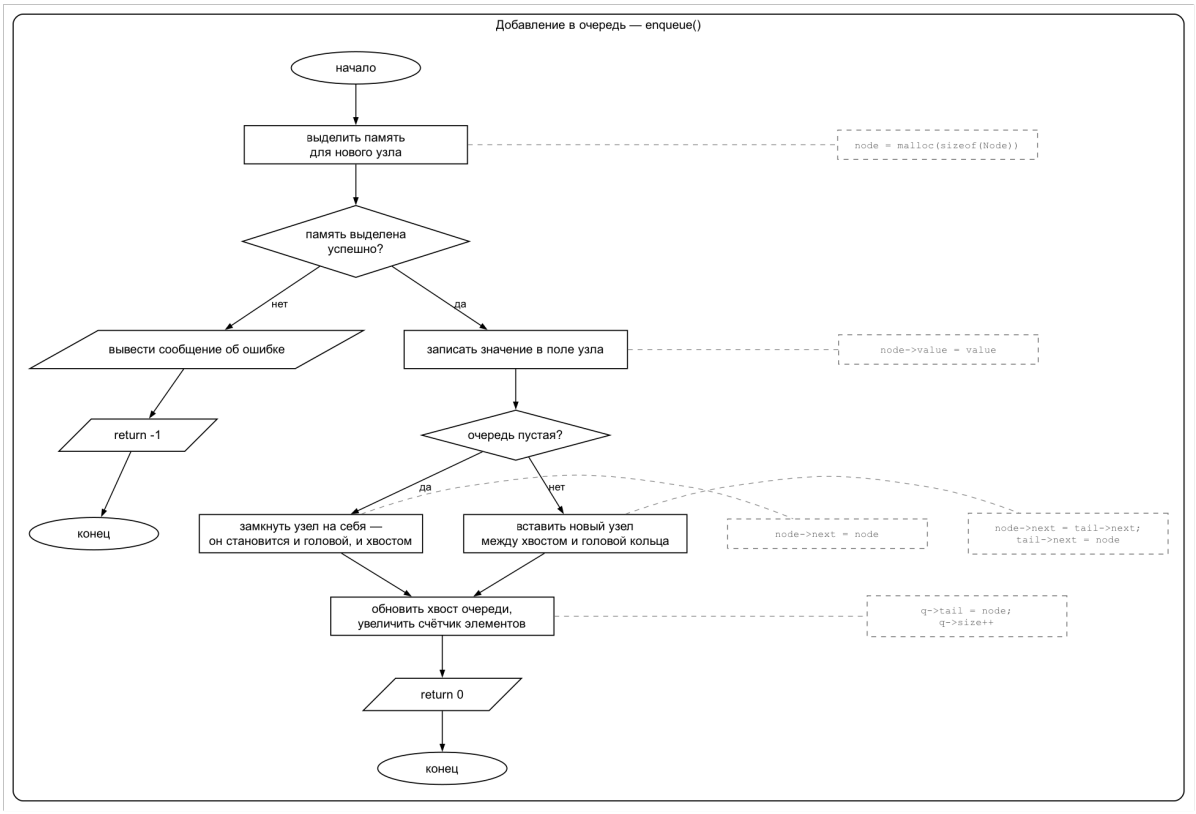
. 2 – (read_file)



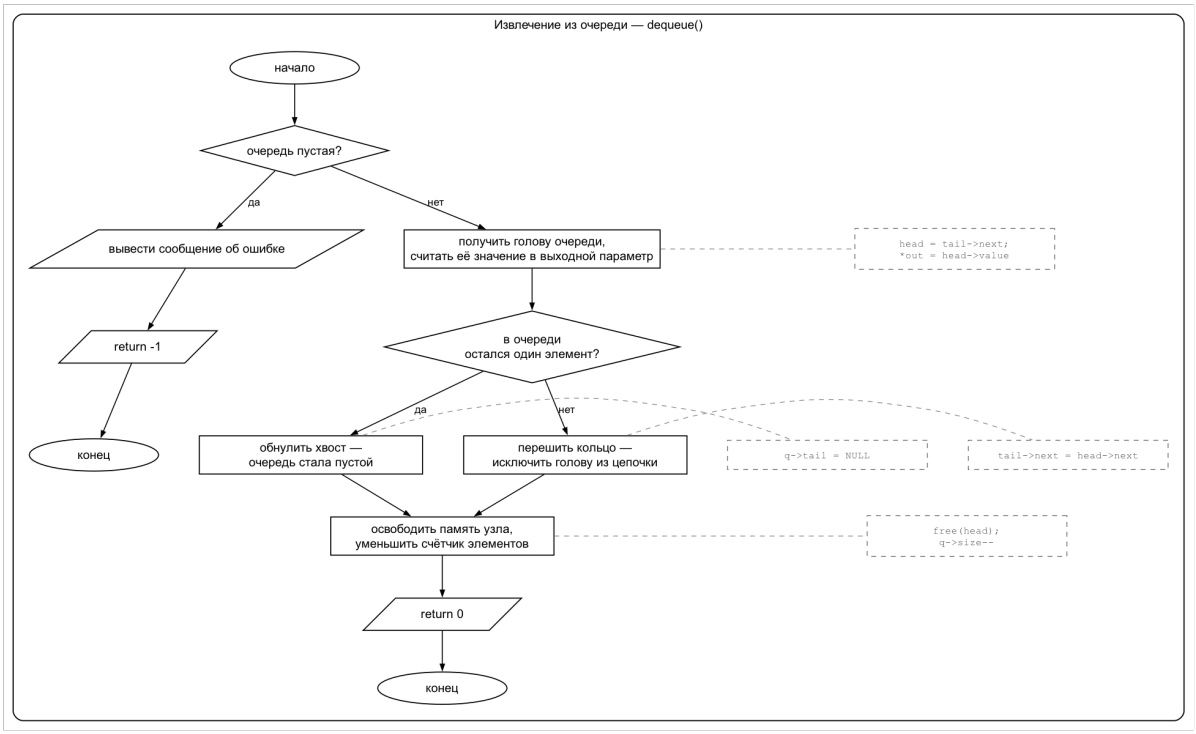
. 3 – (radix_sort)



. 4 – (counting_sort_by_queue)


Добавление в очередь — enqueue()
начало
выделить память
для нового узла
node = malloc(sizeof(Node))
память выделена
успешно?
нет
да
вывести сообщение об ошибке
записать значение в поле узла
node->value = value
return -1
очередь пустая?
да
нет
конец
замкнуть узел на себя —
он становится и головой, и хвостом
вставить новый узел
между хвостом и головой кольца
node->next = node
node->next = tail->next;
tail->next = node
обновить хвост очереди,
увеличить счётчик элементов
q->tail = node;
q->size++
return 0
конец

. 5 – (enqueue)


Извлечение из очереди — dequeue()
начало
очередь пустая?
да
нет
вывести сообщение об ошибке
получить голову очереди,
считать её значение в выходной параметр
head = tail->next;
*out = head->value
return -1
в очереди
остался один элемент?
конец
да
нет
обнулить хвост —
очередь стала пустой
перешить кольцо —
исключить голову из цепочки
q->tail = NULL
tail->next = head->next
освободить память узла,
уменьшить счётчик элементов
free(head);
q->size--
return 0
конец

. 6 – (dequeue)